Sprint 4 – Projecte CRUD amb FastAPI + MongoDB + Frontend

Objectiu

Desenvolupar una **aplicació web completa** que permeti gestionar informació mitjançant **operacions CRUD**. Cada alumne haurà d'escollir **un dels dominis proposats** (vegeu apartat següent).

L'aplicació constarà de:

- **Backend** amb FastAPI.
- **Base de dades** MongoDB (accés amb **PyMongo Async**).
- **Frontend** amb HTML, CSS i JavaScript.

Elecció del domini

Heu d'escollir **una de les següents opcions**:

1. Gestor de tasques

- títol, descripció
- estat (pendent / feta)
- prioritat
- categoria
- persona assignada

2. Gestor de pel·lícules

- títol, descripció
- estat (pendent de veure / vista)
- puntuació (1–5)
- gènere
- usuari

3. Gestor de llibres

- títol, autor
- estat (pendent / llegit)
- valoració
- categoria
- persona

4. Gestor de comandes

- producte, descripció
- estat (pendent / enviada / entregada)
- prioritat
- categoria
- client

5. Gestor d'incidències

- títol, descripció
- estat (oberta / en procés / resolta)
- prioritat
- categoria
- tècnic assignat

Material inicial

Al accedir al github classroom disposem d'una **plantilla base**. El codi i documentació l'heu de lliurar mantenint actualitzat el vostre repo del github classroom:

- ASIX: https://classroom.github.com/a/ULL36zWV
- DAW: https://classroom.github.com/a/IkbHM2gB

Aquesta plantilla està basada en l'**exemple oficial de MongoDB amb FastAPI** que heu estat treballant en classe

- https://github.com/mongodb-developer/mongodb-pymongo-fastapi

Però, a diferència del tutorial de MongoDB anterior, els tests els realitzareu en postman.

Estructura del projecte

Tot i que el codi .py es podria modular, en aquest cas **treballarem amb una estructura simple**, igual que a l'exemple oficial. Tot el backend s'ubica en un únic fitxer (`app.py`), amb l'objectiu de centrar-se en **aprendre CRUD amb FastAPI i MongoDB** abans de **modularitzar el codi**.

El projecte ha de mantenir una **estructura com aquesta**:

```
project/
├── README.md
├── backend/                # FastAPI + MongoDB
│   ├── app.py              # Fitxer principal (tota la lògica)
│   └── requirements.txt    # Dependències
│
├── frontend/           # Interfície web
│   ├── index.html
│   ├── style.css
│   └── app.js
│
└── tests/              # Tests amb Postman
    └── Postman_API_tests.json
```

Fitxer `app.py`

En projectes més complexos, es separaria/modularia, per exemple, la **connexió a MongoDB** en un fitxer a banda, anomenat `database.py`; i, els **models**, en `models.py`.

En el nostre cas, tot el backend l'implementarem dins del fitxer `app.py` per simplificar.

Tot i això, **cal**:

- Afegir **grans comentaris per separar lògica** de connexió, models i endpoints.
- **Documentar clarament cada secció** per facilitar la lectura i localització d'errors.

Un bon exemple seria aquest:

```
import os
from typing import Optional, List

from fastapi import FastAPI, Body, HTTPException, status
from fastapi.responses import Response
from pydantic import ConfigDict, BaseModel, Field, EmailStr
from pydantic.functional_validators import BeforeValidator
from typing_extensions import Annotated

from bson import ObjectId
import asyncio
from pymongo import AsyncMongoClient
from pymongo import ReturnDocument

# ------------------------------------------------------------------------ #
#                          Inicialització de l'aplicació                   #
# ------------------------------------------------------------------------ #
# Creació de la instància FastAPI amb informació bàsica de l'API
app = FastAPI(
    title="Student Course API",
    summary="Exemple d'API REST amb FastAPI i MongoDB per gestionar informació
d'estudiants",
)

# ------------------------------------------------------------------------ #
#                    Configuració de la connexió amb MongoDB               #
# ------------------------------------------------------------------------ #
# Creem el client de MongoDB utilitzant la URL de connexió emmagatzemada
# a les variables d'entorn. Això evita incloure credencials dins del codi.
client = AsyncMongoClient(os.environ["MONGODB_URL"])

# Selecció de la base de dades i de la col·lecció
```

```python
db = client.college
student_collection = db.get_collection("students")

# Els documents de MongoDB tenen `_id` de tipus ObjectId.
# Aquí definim PyObjectId com un string serialitzable per JSON,
# que serà utilitzat als models Pydantic.
PyObjectId = Annotated[str, BeforeValidator(str)]

# ------------------------------------------------------------------------ #
#                            Definició dels models                         #
# ------------------------------------------------------------------------ #
class StudentModel(BaseModel):
    """
    Model que representa un estudiant.
    Conté tots els camps obligatoris i opcional `_id`.
    """
    # Clau primària de l'estudiant.
    # MongoDB utilitza `_id`, però l'API exposa aquest camp com `id`.
    id: Optional[PyObjectId] = Field(alias="_id", default=None)

    # Camps obligatoris de l'estudiant
    name: str = Field(...)
    email: EmailStr = Field(...)
    course: str = Field(...)
    gpa: float = Field(..., le=4.0)

    # Configuració addicional del model Pydantic
    model_config = ConfigDict(
        populate_by_name=True,  # Permet utilitzar alias al serialitzar/deserialitzar
        arbitrary_types_allowed=True,  # Permet tipus personalitzats com ObjectId
        json_schema_extra={
            "example": {
                "name": "Jane Doe",
                "email": "jdoe@example.com",
                "course": "Experiments, Science, and Fashion in Nanophotonics",
                "gpa": 3.0,
            }
        },
    )
```

Requeriments funcionals

L'aplicació ha d'implementar com a mínim les següents funcionalitats:

Operacions bàsiques

- Crear un nou element (C)
- Llistar tots els elements (R)
- Editar un element existent (U)
- Eliminar un element (D)

Gestió d'estat

- Canviar l'estat (segons el domini: feta, vista, resolta, etc.).

Classificació

- Assignar una categoria.
- Assignar una prioritat o valoració.
- Llistar elements filtrats per categoria o prioritat.

Assignació

- Assignar l'element a una persona (usuari, client, tècnic, etc.).

Requeriments tècnics

Backend

- Desenvolupar una **API REST** amb **FastAPI**.
- Implementar **totes les operacions CRUD**.
- Utilitzar **PyMongo async** per a la **connexió amb MongoDB** *(no utilitzar Motor)*.
- **Definir** correctament **models de dades** (**Pydantic**).

Frontend

- Consumir l'API mitjançant JavaScript (`fetch`).
- Interfície funcional per a totes les operacions.
- Estil amb un framework CSS:
  - http://getskeleton.com/
  - https://milligram.io/

Tests

- Utilitzar **Postman** per validar els endpoints
- **Incloure** una col·lecció de **tests bàsics** al **repositori**

Lliurament

- **Data límit (ASIX):** 17 d'abril.
- **Format:** repositori GitHub Classroom.

Presentació (autoavaluació i coavaluació)

- **Data:** 17 d'abril.
- **Format:** en equip (de la mateixa manera que vam fer-ho en el sprint 3).

Criteris d'avaluació

- Correcte funcionament del CRUD (35%)
- Integració frontend-backend (35%)
- Tests amb Postman (20%)
- Organització del projecte (10%)

Proposta Llistat de tasques

https://docs.google.com/spreadsheets/d/1xIn2cNeJVb9_b3qBzc0Eh6svwMvaQucFUIzU-c4s2uM/edit?usp=sharing